Project Title	**Real-Time Object Detection using YOLOv8 (Images & Videos)**
Skills take away From This Project	**Computer Vision fundamentals (bounding boxes, IoU, confidence, NMS)** **Using Ultralytics YOLOv8 for detection (train/val/predict)** **Dataset preparation in YOLO format (images + labels)** **Model evaluation (mAP, Precision, Recall) and error analysis** **Inference on image, video, webcam + saving annotated outputs** **Basic deployment mindset (Streamlit/Flask optional**
Domain	

Problem Statement:

Build an end-to-end Object Detection system using YOLOv8 that can:

1. Train a custom detector on a labeled dataset
2. Evaluate performance using standard detection metrics
3. Run inference on new images/videos/webcam and generate annotated outputs

Business Use Cases:

Retail: detect products, customers, queue length, shelf monitoring

Traffic & Smart City: detect vehicles, helmets, number plates (as objects), congestion analysis

Manufacturing: defect detection, missing component detection

Security: person detection, restricted zone monitoring

Healthcare: detecting medical instruments/objects in imaging workflows

Sports/Media: player/ball detection for analytics

Approach:

Dataset Selection & Understanding

- Choose dataset (public like COCO subset / custom collected)
- Identify class list and detection goal

Data Preparation (YOLO Format)

- Organize folders: `images/train`, `images/val`, `labels/train`, `labels/val`
- Create label `.txt` per image: `class x_center y_center width height` (normalized 0–1)
- Write `data.yaml` with paths + class names

Model Setup (YOLOv8)

- Install dependencies and verify GPU/CPU environment
- Start with a baseline model: `yolov8n.pt` / `yolov8s.pt`

Training

- Train with chosen epochs, image size, batch size
- Track training/validation curves and loss trends

Evaluation & Error Analysis

- Report **mAP50**, **mAP50-95**, Precision, Recall
- Inspect failure cases: false positives, false negatives, small objects

Inference Pipeline

- Predict on images/videos/webcam

- Save outputs: annotated images, output video, inference logs

Optional Enhancements

- Hyperparameter tuning, augmentation adjustments
- Convert/export model (ONNX) or build Streamlit demo

Results:

By the end, learners should produce:

- **A trained YOLOv8 model capable of detecting target classes**
- **A metrics report (mAP, precision, recall) with clear interpretation**
- **Sample predictions on unseen images/videos showing bounding boxes + labels + confidence**
- **A short conclusion on what worked, what failed, and how to improve**

Project Evaluation metrics:

mAP@0.5 and **mAP@0.5:0.95**

Precision and **Recall**

Confusion patterns (class-wise performance)

Inference speed (FPS) on CPU/GPU (optional)

Quality of output demo (clean visual results + reproducible pipeline)

Technical Tags:

`YOLOv8`, `Object Detection`, `Computer Vision`, `Deep Learning`, `PyTorch`, `Ult`

Data Set: **YOLO V8 vehicle detection Notebook**

Option A (Recommended for beginners): COCO128

- Small curated dataset used commonly for quick YOLO experiments
- Format: images + YOLO labels

Option B (Custom Dataset):

- Collect 200–2000 images for your target classes
- Annotate using LabelImg / Roboflow / CVAT

Data Set Explanation:

Each image has one matching label file (`.txt`)

Each row in label file:
`class_id x_center y_center width height` (all normalized)

Preprocessing required:

- Remove corrupt images
- Ensure label-image matching
- Split into train/val (e.g., 80/20)
- Validate class IDs and YAML config correctness

Project Deliverables:

Source Code / Notebook

- Training script/notebook
- Inference script for image + video + webcam

Dataset YAML + folder structure screenshot or tree

Trained weights (best.pt) or link to drive/github release

Evaluation Report

- metrics table + brief analysis + sample predictions

Demo Outputs

- 5–10 annotated images + 1 annotated video clip

README.md

- Setup steps, commands, how to reproduce results

Project Guidelines:

Use a clean repo structure:

- `data/`, `notebooks/`, `src/`, `runs/`, `outputs/`, `README.md`

Keep parameters configurable (epochs, imgsz, batch, model size)

Commit meaningful messages, maintain version control (GitHub)

Document environment (Python version, ultralytics version)

Avoid mixing raw dataset with code repo if dataset is large (use links)

Timeline:

Define the project timeline, including milestones and deadlines.

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.
Note: Book the slot at least before 12:00 Pm on the same day

Timing: Monday-Saturday (3:30PM to 4:30PM)

Booking link :https://forms.gle/XC553oSbMJ2Gcfug9

For DE/BADM project/class topic doubt slot clarification session:

Booking link : https://forms.gle/NtkQ4UV9cBV7Ac3C8

Session timing:

For DE: 04:00 pm to 5:00 pm every saturday
For BADM 05:00 to 07:00 pm every saturday

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.
Note: This form will Open only on Saturday (after 2 PM) and Sunday on Every Week

Timing:

For BADM and DE
Monday-Saturday (11:30AM to 1:00PM)

For DS and AIML
Monday-Saturday (05:30PM to 07:00PM)

Booking link : https://forms.gle/1m2Gsro41fLtZurRA